

自指、对角化与形式系统的边界

罗素悖论、哥德尔不完备性与图灵不可判定性

一份从直觉到技术骨架的讲义

阅读目标

三个结果看似分属集合论、证明论和计算理论，却共享一种结构：系统把关于自身的描述变成内部对象，再让这个对象作用于自身，并在关键位置实行反转。本讲义先解释这条共同主线，再分别重构图灵与哥德尔的思想。

目录

1 三个结果，三种边界	2
2 共同结构：把系统描述也变成内部对象	2
2.1 为什么这会带来危险？	2
2.2 同一个对角骨架	3
3 图灵机：先定义“机械计算”	3
3.1 通用机：程序也可以成为数据	4
4 图灵的反转：为什么停机不可判定	4
5 为什么哥德尔比它们更难理解？	5
5.1 为了自指，需要一个严格的“代指”	5
6 哥德尔句：先造模板，再填入模板自己的名字	6
6.1 第一步：定义对角代入	6
6.2 第二步：设计所需模板	6
6.3 第三步：把模板自己的编码填入模板	6
7 为什么必须自指？为什么选择“不可证明”？	7
8 从哥德尔句到两个不完备性定理	7
8.1 第一不完备性定理	7
8.2 第二不完备性定理	8
9 统一视角：矛盾究竟发生在哪里？	8

1 三个结果，三种边界

罗素、哥德尔与图灵的结论并不相同，但它们都在否定一种过强的“万能设想”。

	被否定的万能设想	最终结论
罗素	任意性质都能无条件地确定一个集合	不存在这样的无限制集合构造原则
哥德尔	存在一致、有效而且能判定全部算术句子的理论	这样的理论必然有既证不出也反驳不了的句子
图灵	存在能够判定所有程序行为的算法	不存在万能的停机判定程序

这里没有得到“某件事既不能为真也不能为假”。罗素的矛盾说明假设的集合不能存在；哥德尔的“不可判定”首先是相对于理论的不可证明与不可反驳；一台具体机器是否停机也有确定答案，只是没有一种算法能统一给出所有答案。

2 共同结构：把系统描述也变成内部对象

三个问题的共同点，可以用一句话概括：

共同起点

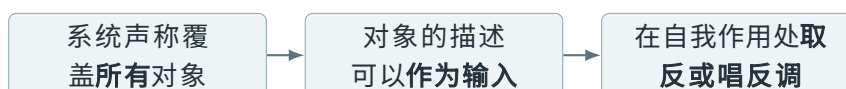
系统不仅处理普通对象，还能够把“对系统自身的描述”转化成同类对象，于是描述可以重新作为输入交给系统自己。

三个领域分别通过不同方式完成这种“内化”：

- **罗素**：集合的元素仍然可以是集合，所以“某个集合是否属于自身”本来就是集合论内部的问题。
- **哥德尔**：公式和证明原本是语言对象；哥德尔编码把它们变成自然数，于是算术能够借助数字谈论自己的公式与证明。
- **图灵**：程序是一段有限符号串，可以编码成自然数；自然数又能作为程序输入，所以程序可以读取、模拟甚至处理自己的描述。

2.1 为什么这会带来危险？

严格地说，并不是“绝对不能”把系统的描述当成内部对象。程序作为数据、公式编码成数字，都是现代逻辑与计算机科学最有力量的思想。真正危险的是以下条件同时出现：



这时便可以构造一个对象，专门在系统检查它自己的位置上反抗系统的判断。若系统仍坚持自己“没有遗漏”，就会与自身冲突。

因此，现代形式理论通常通过分层、限制集合构造、区分对象语言与元语言，或者放弃某种“全能判定能力”来避免矛盾。

2.2 同一个对角骨架

把所有对象排成

$$A_0, A_1, A_2, \dots$$

并令 $R(i, j)$ 表示“第 i 个对象对输入 j 作出肯定反应”。对角线考察的是

$$R(0, 0), R(1, 1), R(2, 2), \dots,$$

而反对角对象被定义为

$$D(n) \iff \neg R(n, n).$$

记忆公式

编号 + 自用 + 反转 = 对角化的核心动力

3 图灵机：先定义“机械计算”

图灵面对希尔伯特的判定问题，首先必须回答：所谓“算法”究竟是什么？他的办法不是研究某一台现实机器，而是观察一个严格按规则计算的人，并删去一切与计算能力无关的细节。

人工计算员	图灵机中的抽象
草稿纸与中间结果	分成格子的纸带
目光当前所在位置	读写头
正在进行哪一步、暂时记住什么	有限状态
按规则读、写、移动	有限的转换规则表

一次基本操作只依赖“当前状态”和“当前符号”：

$$(q, a) \mapsto (q', b, L/R).$$

它表示：机器处于状态 q 、读到符号 a 时，写下 b ，向左或向右移动，并进入状态 q' 。复杂步骤可以拆成许多简单步骤；这可能变慢，却不会改变原则上能否完成计算。

3.1 通用机：程序也可以成为数据

每台图灵机都是一张有限规则表，因此可以编码为自然数 $\lceil M \rceil$ 。图灵构造出一台通用机 U ：

$$U(\lceil M \rceil, x) \text{ 模拟 } M(x).$$

同一台机器只要换一段编码，就能模拟不同机器。这是现代通用计算机的逻辑原型：**硬件保持不变，程序作为数据被读取和执行。**

通用模拟机 U 确实存在：它能模拟任何已有算法。

万能判定器 H 不存在：它不能预先判断任意程序在任意输入上会不会停机。

“能执行所有算法”不等于“能回答关于所有算法的一切问题”。

4 图灵的反转：为什么停机不可判定

把所有程序编号为

$$M_0, M_1, M_2, \dots$$

并考察对角线

$$M_0(0), M_1(1), M_2(2), \dots$$

这里 $M_n(n)$ 表示：把第 n 台机器自己的编码 n 输入它自身。

现在暂时假设存在万能判定器 H ，它能正确回答“ $M_n(n)$ 是否停机”。利用 H 构造一台反转机器 B ：

反转机器 B

输入 n 后：

1. 若 H 判断 $M_n(n)$ 会停机， B 就故意永远运行；
2. 若 H 判断 $M_n(n)$ 不会停机， B 就立即停机。

换言之， B 专门与 H 对对角输入的判断唱反调。

设 B 自己的编码为 b ，再运行 $B(b)$ ：

- 若 H 判断 $B(b)$ 会停机， B 按定义就不停机；
- 若 H 判断 $B(b)$ 不会停机， B 按定义就停机。

无论 H 怎样回答都会错。因此矛盾不在于 $B(b)$ “既停机又不停机”，而在于我们假设了一个永远正确的 H 。正确结论是：

图灵的边界

不存在能够判定所有程序在所有输入上是否停机的算法。

5 为什么哥德尔比它们更难理解？

困难在哪里？

集合可以包含集合，程序代码可以作为程序输入；但是算术公式不能被代入自身。更准确地说，算术公式中的变量位置只能代入数，不能直接塞进另一个公式。

哥德尔的目标是构造一个形式上完全合法的句子 G ，它的直观内容是：

“这句话在理论 T 中不可证明。”

这句话在自然语言里不难理解，困难在于：自然语言中的“这句话”依靠语境指向自己，而形式系统中没有这样含糊的指示语。因此，必须用一种科学、机械、无歧义的方法，让这个句子真正指向自身。

5.1 为了自指，需要一个严格的“代指”

为了完成自指，我们希望找到同样一个东西，既能代指“这句话”，又能成为“这句话在 T 中不可证明”这一陈述所谈论的对象。

真正的要求

这个“代指”不能只靠我们从自然语言中体会出来，而必须有一套明确的构造规则，使理论能够机械地识别它究竟指向哪个句子。

由此，编码就自然出现了：哥德尔给符号、公式和证明分配自然数编码，让一个句子获得能够在算术中使用的“名字”。于是产生三个层次：

编码产生的三个层次

1. 句子本身： G ；
2. 句子 G 的编码，也就是它在算术中的名字： $\ulcorner G \urcorner$ ；
3. 关于这个编码是否存在证明的算术公式： $\text{Prov}_T(\ulcorner G \urcorner)$ 。

由于证明是有限符号序列，“ p 是否编码了对句子 q 的合法证明”可以机械检查，因此可以定义

$$\text{Prov}_T(q) \equiv \exists p \text{ Proof}_T(p, q).$$

它表示：“存在一个自然数 p ，编码了理论 T 中对编码为 q 的句子的证明。”

现在还剩最后一个问题：怎样让上面第三层中的编码，恰好就是这个公式最终形成的句子自己的编码？为此，需要定义一个严格的**对角代入函数** d 。

从目标到技术构造

目标句 $\rightarrow$ 精确代指 $\rightarrow$ 编码 $\rightarrow$ 三个层次 $\rightarrow$ 对角代入与自指

6 哥德尔句：先造模板，再填入模板自己的名字

有了编码，还缺最后一步：怎样让一个句子指向自己的编码？直觉上可以把它理解为“先写一个带空格的模板，再把模板自己的编号填进去”。

6.1 第一步：定义对角代入

定义 $d(n)$ ：

$d(n)$ 是这样一个句子的编码：先找到编码为 n 的单变量公式，再把数字 n 代入这个公式的自由变量。

因此，若 $e = \ulcorner \theta(x) \urcorner$ ，那么

$$d(e) = \ulcorner \theta(e) \urcorner.$$

d 的作用是把“模板的编码”转换成“模板填入自身编码后所得句子的编码”。

6.2 第二步：设计所需模板

定义

$$\theta(x) \equiv \neg \text{Prov}_T(d(x)).$$

它的自然语言意思是：

把编码为 x 的公式代入 x 后，所得句子在理论 T 中不可证明。

此时 $\theta(x)$ 还只是一个带有变量 x 的模板，并未直接声称自己不可证明。

6.3 第三步：把模板自己的编码填入模板

令

$$e = \ulcorner \theta(x) \urcorner, \quad G = \theta(e).$$

由于编码为 e 的公式正是 $\theta(x)$ ，所以

$$d(e) = \ulcorner \theta(e) \urcorner = \ulcorner G \urcorner.$$

再展开 $G = \theta(e)$ 的定义：

$$\begin{aligned} G &\leftrightarrow \neg \text{Prov}_T(d(e)) \\ &\leftrightarrow \neg \text{Prov}_T(\ulcorner G \urcorner). \end{aligned}$$

形式理论内部可以验证这一构造，因此得到

$$T \vdash G \leftrightarrow \neg \text{Prov}_T(\ulcorner G \urcorner).$$

哥德尔句的直觉内容

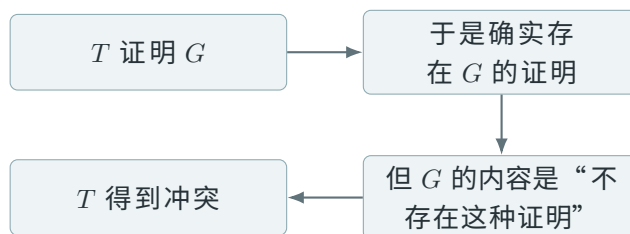
理论 T 能够证明： G 成立，当且仅当 G 在 T 中不可证明。通常把它简称为：“ G 说自己在 T 中不可证明。”

这不是循环定义。我们没有先知道 G 的编码再把它写进 G ；而是先构造模板 $\theta(x)$ ，再把模板自己的编码 e 填入模板。自我指涉是这个有限机械过程的结果。

7 为什么必须自指？为什么选择“不可证明”？

哥德尔要否定的是：一个一致而有效的理论能够判定所有算术句子。为此，他需要一个句子，使理论一旦证明它，就会反过来破坏自身的一致性。

普通句子“另一个句子 A 不可证明”并不能做到这一点。即使它被证明了，也未必反过来约束它自己。只有让句子谈论自己的可证明性，证明行为才会反馈到句子的内容上。



所以只要 T 一致，就不能有

$$T \vdash G.$$

哥德尔没有使用“这句话是假的”，而是使用“这句话不可证明”，原因在于：

不可证明 $\neq$ 为假.

“这句话是假的”会形成直接的说谎者矛盾；“这句话不可证明”则允许一种稳定情况：句子为真，但理论无法证明它。哥德尔把会爆炸的真假悖论，转化成了揭示证明边界的不完备性结果。

8 从哥德尔句到两个不完备性定理

8.1 第一不完备性定理

如果 T 是有效公理化、包含足够算术而且一致的，那么它不能证明自己的哥德尔句 G 。在标准自然数解释中，既然确实没有 T -证明， G 所说的内容就是正确的。

要进一步得到“ T 也不能证明 $\neg G$ ”，哥德尔原始证明使用了 ω -一致性等较强条件；罗瑟后来修改句子，使普通一致性就足以得到一个既不能证明也不能反驳的句子。

第一不完备性定理

任何一致、有效公理化并包含足够算术的理论，都不可能判定所有算术句子。

这里的“不可判定”是语法意义的：

$$T \not\vdash G, \quad T \not\vdash \neg G.$$

它不是说 G 没有真值。一个确定的经典模型仍然使它为真或为假；在标准自然数模型中，通常的哥德尔句为真。

8.2 第二不完备性定理

把“ T 是一致的”在算术内部写成

$$\text{Con}(T) \equiv \neg \text{Prov}_T(\ulcorner 0 = 1 \urcorner).$$

理论内部能够形式化第一定理的关键推理，得到

$$T \vdash \text{Con}(T) \rightarrow G.$$

如果 T 能证明 $\text{Con}(T)$ ，它就能进一步证明 G ；但一致的 T 不能证明 G 。因此

$$T \not\vdash \text{Con}(T).$$

这并不是说一致性永远不能被证明，而是说一个足够强的一致理论不能仅靠自身证明其标准形式化的一致性；更强的理论仍可能证明较弱理论的一致性。

9 统一视角：矛盾究竟发生在哪里？

	自身描述如何进入系统	反转动作	矛盾迫使我们放弃什么
罗素	集合可以集合为元素	$x \in x$ 被改成 $x \notin x$	无限制的集合构造
哥德尔	公式和证明被编码为自然数	“可证明”被改成“不可证明”	一致有效理论的完全性
图灵	程序编码可作为程序输入	对停机预测采取相反行为	万能停机判定算法

真正导致问题的不是自我指涉本身，而是：系统能够处理自己的描述，又声称自己能对所有这类对象作出无遗漏、无错误的总体判断，并允许在自我应用处实行否定性反转。

因此，三个结果的共同教训不是“自指必须禁止”，而是：

总结

当一个系统强到能够描述自身时，关于系统能力的“绝对总体”主张就必须非常谨慎。对角化能够构造一个对象，使它在系统检查自身的位置上专门逃离系统的判断。为了保持一致，系统必须承认某种边界。

本讲义以直觉重构为主；哥德尔编码、对角引理的可表示性证明以及可证明性条件属于下一层技术细节。